

Actividad 10: Road to Hall Of Fame

Laboratorio: File path traversal, traversal sequences stripped with superfluous URL-decode

Asignatura: Seguridad Informática

Realizado por el alumno: Manuel Darío Trejo Salazar

Herramienta utilizada: Burp Suite Community Edition

1.- Nombre del laboratorio

File path traversal, traversal sequences stripped with superfluous URL-decode

URL del laboratorio:

<https://portswigger.net/web-security/file-path-traversal/lab-superfluous-url-decode>

The screenshot shows the PortSwigger lab interface. At the top, the title 'Lab: File path traversal, traversal sequences stripped with superfluous URL-decode' is displayed in a light blue font. Below the title, there is a 'PRACTITIONER' badge and a 'LAB' button with a flag icon. To the right of the 'LAB' button is a 'Not solved' status indicator. A red share icon is located in the top right corner. The main text describes the lab: 'This lab contains a path traversal vulnerability in the display of product images. The application blocks input containing path traversal sequences. It then performs a URL-decode of the input before using it. To solve the lab, retrieve the contents of the `/etc/passwd` file.' Below this text is an orange button with a flask icon and the text 'ACCESS THE LAB'. At the bottom, there are two expandable sections: 'Solution' and 'Community solutions', each with a light blue icon and a downward arrow.

Lab: File path traversal, traversal sequences stripped with superfluous URL-decode

PRACTITIONER

LAB Not solved

This lab contains a path traversal vulnerability in the display of product images.

The application blocks input containing path traversal sequences. It then performs a URL-decode of the input before using it.

To solve the lab, retrieve the contents of the `/etc/passwd` file.

ACCESS THE LAB

Solution

Community solutions

2.- Nivel

Practitioner (Intermedio).

Este laboratorio presenta un escenario donde la aplicación web intenta bloquear los ataques de Path Traversal eliminando secuencias ../, pero introduce una nueva vulnerabilidad al decodificar la entrada de forma superflua (más veces de lo necesario), lo que permite evadir la protección mediante doble codificación URL (Double URL Encoding).

3.- Tipo de explotación

File Path Traversal (Arbitrary File Read) con evasión de filtros mediante Double URL Encoding.

Por lo general, los servidores web (como Apache, Nginx o IIS) decodifican automáticamente la URL de una petición HTTP antes de pasarla a la aplicación web (backend). Si la aplicación web incluye código que vuelve a decodificar explícitamente los datos de entrada de forma manual, se crea una discrepancia peligrosa.

Un atacante puede aplicar una "doble codificación URL" a su payload malicioso. Cuando el servidor web recibe la petición, realiza la primera decodificación. El filtro de seguridad (WAF o código de sanitización) analiza la cadena resultante, y al seguir estando codificada parcialmente, no detecta ninguna firma maliciosa (no encuentra la secuencia literal ../).

Sin embargo, después de pasar la validación con éxito, la aplicación vuelve a decodificar la cadena (decodificación superflua), devolviendo el payload malicioso a su forma original (../) justo antes de pasarlo a la API del sistema de archivos, ejecutando así la vulnerabilidad.

Consecuencias: Lectura de archivos sensibles del sistema operativo y evasión efectiva de Firewalls de Aplicaciones Web (WAF) o filtros de sanitización implementados en el código.

4. Objetivo del laboratorio

Demostrar la explotación de una vulnerabilidad de File Path Traversal evadiendo un filtro de seguridad. Esto se logra ofuscando el parámetro filename mediante una técnica de doble codificación URL (%252e%252e%252f) para reconstruir el salto de directorio en el backend, con el fin de leer el contenido del archivo /etc/passwd del servidor y obtener la confirmación de "Lab solved".

5. Explicación técnica de la vulnerabilidad

La aplicación web utiliza el parámetro filename para cargar imágenes:

GET /image?filename=71.jpg HTTP/2

La defensa de la aplicación consta de dos partes:

Elimina explícitamente las secuencias de recorrido (../).

Posteriormente (y de forma insegura), decodifica la URL de la entrada del usuario antes de utilizarla para acceder al sistema de archivos.

Si enviamos el payload estándar ../../etc/passwd, el filtro lo elimina y la explotación falla.

Si enviamos el payload con codificación URL simple (%2e%2e%2f), el servidor web frontend lo decodifica automáticamente a ../ antes de que llegue a la aplicación, por lo que el filtro del backend también lo detecta y lo elimina.

La falla técnica radica en la decodificación manual posterior al filtrado. Al enviar una doble codificación (%252e%252e%252f):

Paso 1 (Servidor Web): Decodifica los %25 (que es el símbolo % en URL encoding). La cadena se convierte en %2e%2e%2f.

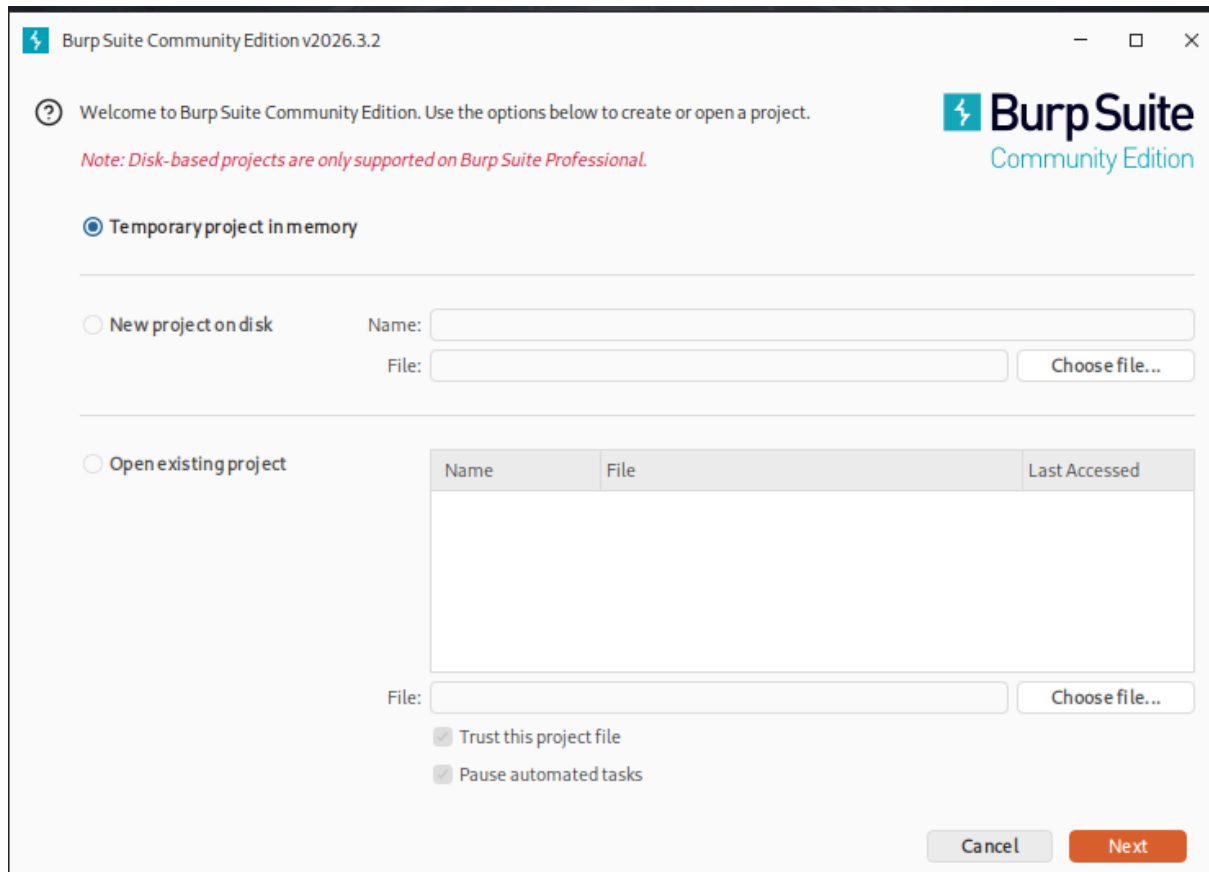
Paso 2 (Filtro): El filtro de seguridad busca la cadena literal ../. Como la entrada actual es %2e%2e%2f, el filtro no detecta peligro y permite que continúe el flujo.

Paso 3 (Decodificación Superflua): La aplicación decodifica explícitamente la cadena %2e%2e%2f transformándola en ../.

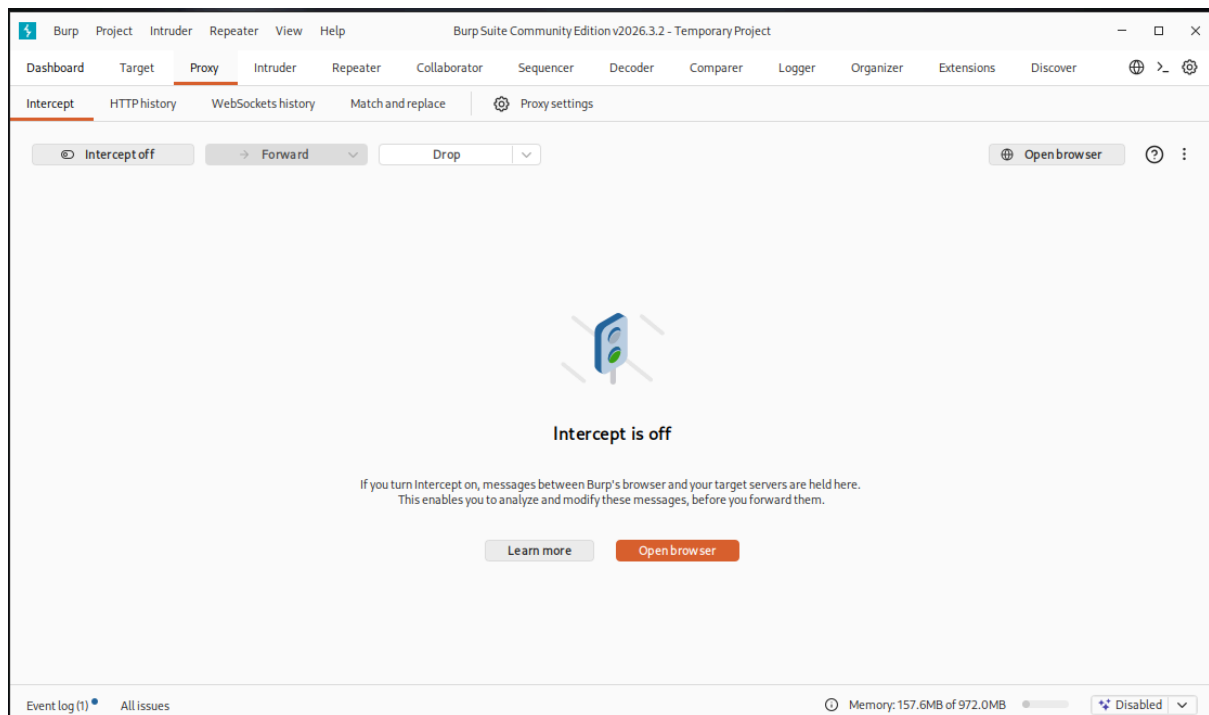
Paso 4 (Ejecución): El sistema de archivos recibe la ruta con los saltos de directorio procesados y devuelve el archivo no autorizado.

6. Procedimiento paso a paso

Se inicia Burp Suite Community Edition desde la máquina virtual, seleccionando en las configuraciones iniciales, Temporary project in memory y posteriormente Use Burp defaults.



Con Burp abierto en la pestaña Proxy > Intercept, se abre el navegador integrado y se accede a la URL del laboratorio proporcionada por PortSwigger.



En Burp Suite se desactiva temporalmente el interceptor ("Intercept is off") para permitir la carga completa de la página de la tienda.

Se navega a la página de un producto (clic en "View details"), lo que dispara la petición que carga la imagen.



File path traversal, traversal sequences stripped with superfluous URL-decode

LAB Not solved

[Back to lab description >>](#)

[Home](#)

WE LIKE TO
SHOP 



Waterproof Tea Bags

★★★★★ \$44.07

[View details](#)



High-End Gift Wrapping

★★★★★ \$54.65

[View details](#)



Real Life Photoshopping

★★★★★ \$67.31

[View details](#)



Mood Enhancer

★★★★★ \$11.86

[View details](#)

Se busca en el historial (pestaña Proxy > HTTP history) la petición GET que carga la imagen (GET /image?filename=...) y se selecciona con clic derecho → "Send to Repeater".

El módulo Repeater permite editar y reenviar la petición observando las respuestas del servidor en tiempo real.

The screenshot shows the Burp Suite interface with the Proxy tab selected. The HTTP history table lists several requests. The third request, a GET to `https://0a9c001b049e5dd4808585ba00d20082.web-security-academy.net/image?filename=29.jpg`, is selected. A right-click context menu is open over this request, showing options like 'Add to scope', 'Forward', 'Drop', 'Add notes', 'Highlight', 'Don't intercept requests', 'Do intercept', 'Scan', 'Send to Intruder', 'Send to Repeater', 'Send to Sequencer', 'Send to Organizer', 'Send to Comparer', and 'Request in browser'. The 'Request' pane at the bottom shows the raw HTTP request details.

Time	Type	Direction	Method	URL
11:05:04 ...	WS	→ To server		https://0a9c001b049e5dd4808585ba00d20082.web-security-academy.net/aca
11:05:32 ...	HTTP	→ Request	GET	https://0a9c001b049e5dd4808585ba00d20082.web-security-academy.net/pro
11:05:49 ...	HTTP	→ Request	GET	https://0a9c001b049e5dd4808585ba00d20082.web-security-academy.net/image?filename=29.jpg
11:05:49 ...	HTTP	→ Request	GET	https://0a9c001b049e5dd4808585ba00d20082.web-security-academy.net/image?filename=29.jpg

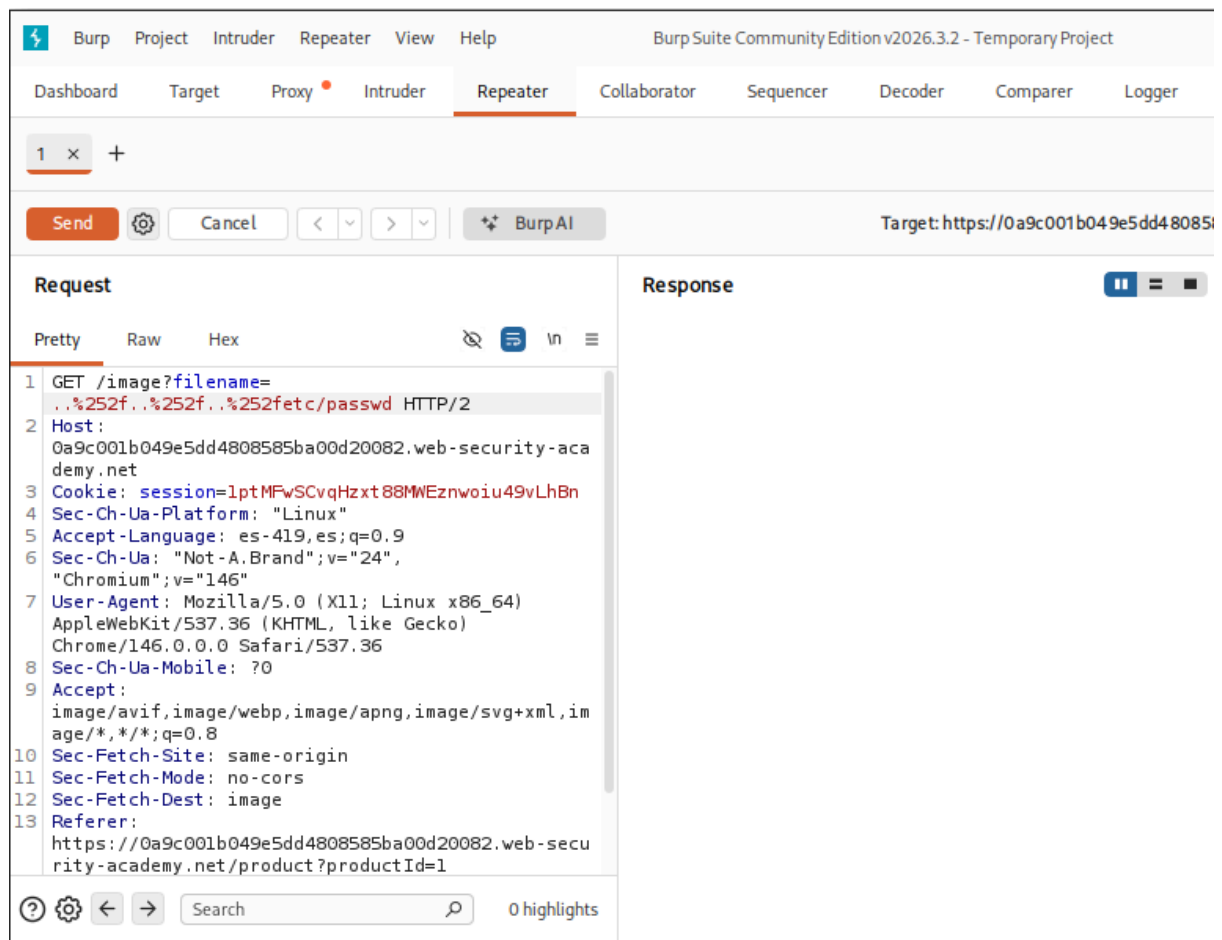
Request

Pretty	Raw	Hex
1	GET /image?filename=29.jpg	HTTP/
2	Host: 0a9c001b049e5dd4808585ba00d20082.web-security-academy.net	
3	Cookie: session=1ptMFwSCvqHzxt88	
4	Sec-Ch-Ua-Platform: "Linux"	
5	Accept-Language: es-419,es;q=0.9	
6	Sec-Ch-Ua: "Not-A.Brand";v="24",	
7	User-Agent: Mozilla/5.0 (X11; Li	
	Safari/537.36	

En la línea 1 del request, dentro de la pestaña Repeater, se modifica el valor del parámetro filename, inyectando el payload con doble codificación URL:

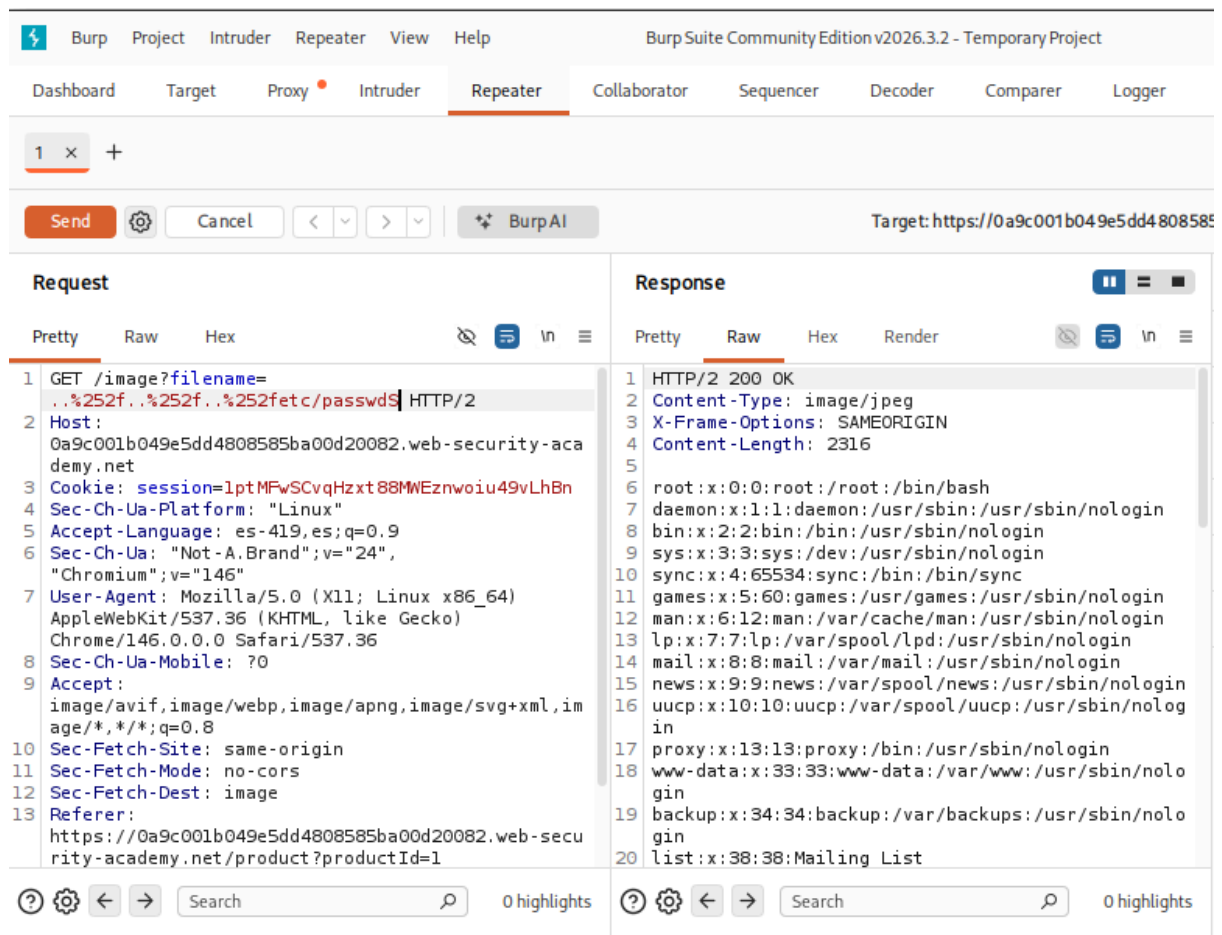
GET

/image?filename=%252e%252e%252f%252e%252e%252f%252e%252e%252f
etc/passwd HTTP/2

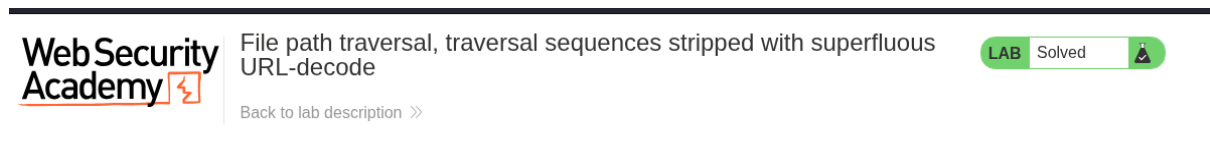


Se presiona "Send".

En el panel Response se observa una respuesta HTTP/2 200 OK. En el cuerpo de la respuesta se visualiza el contenido completo del archivo /etc/passwd del servidor, validando que el filtro fue engañado exitosamente por la codificación:



Se regresa a la pestaña del laboratorio en el navegador y se recarga la página (F5). Apareció el banner verde: LAB Solved! (*Agregar captura de pantalla del banner LAB Solved - "Confirmación de Lab Solved"*)



7. Explicación del payload

Payload utilizado:

`%252e%252e%252f%252e%252e%252f%252e%252e%252fetc/passwd`

Este payload es la representación de `../../../../etc/passwd` codificada dos veces mediante URL Encoding (Hexadecimal).

- Codificación simple:

- Punto `.` = `%2e`

- Barra / = %2f
- Secuencia ../ = %2e%2e%2f
- Doble codificación: Para codificar esto por segunda vez, se codifica el símbolo de porcentaje (%).
 - Símbolo % = %25
 - Por lo tanto, %2e se convierte en %252e
 - Y %2f se convierte en %252f
 - Secuencia doblemente codificada: %252e%252e%252f
 - Al repetir este bloque tres veces (%252e%252e%252f%252e%252e%252f%252e%252e%252f), nos aseguramos de escalar suficientes niveles en el árbol de directorios para llegar a la raíz. Finalmente se concatena la ruta del archivo deseado etc/passwd (el cual no es estrictamente necesario codificar porque el filtro no busca estas palabras).

8. Mitigación recomendada

- Evitar decodificaciones manuales innecesarias: Confiar en que los frameworks modernos y los servidores web ya realizan la normalización y decodificación URL de las peticiones entrantes de forma estándar. No incluir funciones como `urldecode()` a nivel de código de aplicación a menos que esté estrictamente justificado, ya que introducen inconsistencias.
- El orden de las operaciones es crítico: Si es absolutamente necesario decodificar la entrada manualmente, la validación de seguridad (filtros, listas blancas) siempre debe realizarse al final, después de que se hayan completado todas las decodificaciones. Jamás se debe validar una cadena y luego decodificarla.

- Uso de referencias indirectas (Recomendación óptima): Al igual que en todos los casos de Path Traversal, la mejor defensa es evitar referenciar directamente los nombres de archivo en los parámetros HTTP. Utilizar en su lugar un identificador único (ID de base de datos) para mapear internamente la imagen que se desea mostrar.
- Canonicalización estricta: Resolver siempre la ruta final absoluta de la solicitud y verificar que esté contenida dentro del directorio restringido antes de abrir el recurso (ej. `startsWith("/var/www/images/")`).